Biotechnology and Bioengineering

WILEY

BIOTECHNOLOGY AND BIOENGINEERING

ARTICLE OPEN ACCESS

Dynamic, Unconstrained Optimization of Secreted Enzyme Production in Fed-Batch Fermentation Using Reinforcement Learning

Sai Harish Uthravalli[1] | Sakib Ferdous[2] | J. Michael Hess[3] | Nigel F. Reuel[2]

[1]Department of Computer Science, College of Liberal Arts and Sciences, Iowa State University, Ames, Iowa, USA | [2]Department of Chemical and Biological Engineering, College Engineering, Iowa State University, Ames, Iowa, USA | [3]Novonesis, North America, Franklinton, North Carolina, USA

Correspondence: Nigel F. Reuel (reuel@iastate.edu)

Received: 5 September 2025 | **Revised:** 15 April 2026 | **Accepted:** 1 May 2026

Funding: BioMADE; Schmidt Futures, Grant/Award Number: T-SF01-A-09-0034

Keywords: Aspergillus niger | fed-Batch fermentation | fermentation | glucoamylase | in-silico simulation | Monod model | PPO | reinforcement learning | SAC

ABSTRACT

Reinforcement learning (RL) has been used to control a wide range of dynamic processes, especially ones that are too complex to model well or have stochastic environmental perturbations. Fed-batch fermentations are subject to changes in starting cell growth rates and process variations that can affect cell growth and secreted target production. RL has been shown on digital environments of fermentation to control known setpoints (such as temperature) but has yet to be demonstrated for unconstrained product maximization. In this work we develop a fed-batch fermentation model (digital twin) of *Aspergillus niger* secreting glucoamylase using the Monod model, known literature parameters, and assumed constants to align with typical production values. An RL agent is trained on this environment to evaluate types of algorithms (Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC)), rate of learning, and effects of process perturbations. State variables fed to the model include run time, cell concentration, and measured enzyme activity in the fermentation broth, with the objective of maximizing the enzyme activity. It is found that SAC outperforms PPO, achieving 77.7% of the maximum quality with 200 training episodes and 95% at the 2400th episode, compared to PPO which achieves 80% of the max reward after 2912 episodes of training. The RL controller is benchmarked against a traditional, model-free controller that used Bayesian optimization to discover the optimal feed rate for a given cell type. The traditional controller can be implemented with fewer training runs; however, it is not as robust when exposed to variations in starting cell growth or process perturbations including faulty feed or cooling pumps. In all cases, the RL controller can maintain higher enzyme production, despite changes in the process. Finally, the RL controller is exposed to new cell types (*in silico*) to determine the experimental cost of updating the trained model with real bioreactor runs. Surprisingly, we found that with no updates the model can perform well across a wide range of new cell types, and that by retraining the quality of performance improves. These results indicate that an *in silico* trained RL agent can be updated with an array of fermentation experiments to provide robust fermentation control.

1 | Introduction

Humans have employed fermentation for thousands of years to alter and preserve foods, with evidence indicating its origins as early as 10,000 BC (Mannaa et al. 2021a), likely first occurring accidentally as microbes acting on leftover food. In these early ages, fermentation stages were recognized and controlled primarily through taste and feel (Yerolla et al. 2023), as there were no advanced sensors to monitor microbial growth. Qualities

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2026 The Author(s). *Biotechnology and Bioengineering* published by Wiley Periodicals LLC.

such as feel, touch, and texture served as indicators of final product quality (Foegeding and Drake 2007). A deep familiarity with the process was crucial for achieving consistent results. Over time, methods like back-slopping (Kim et al. 2018), where a portion of a previously fermented batch was used to inoculate fresh ingredients, helped maintain microbial stability. The development of starter cultures later improved standardization, leading to enhanced safety, flavor, and reproducibility (Mannaa et al. 2021b).

Modern fermentation processes are broadly classified into batch, fed batch, and continuous modes, each with distinct advantages and challenges. Batch fermentation is a closed system where all nutrients are added at the beginning, and no additional input or output occurs until the process completes. While simple to operate and less prone to contamination, it suffers from low productivity and long downtime for cleaning and setup (Behl et al. 2023). Fed-batch fermentation, widely used in biomanufacturing, allows for controlled nutrient feeding during fermentation, enhancing product yield and reducing substrate inhibition. However, it requires complex feeding strategies to optimize production (Behl et al. 2023). Continuous fermentation maintains a steady-state operation by simultaneously feeding fresh media and removing spent culture, leading to higher productivity but posing greater risks of contamination and genetic instability.

To optimize these fermentation modes, various modern control strategies have been implemented, moving beyond the historic taste and feel strategies (Chai et al. 2022). For example, feed can be maintained by Proportional-Integral-Derivative (PID) control to a specific set point with input of pH, temperature, and dissolved oxygen sensors, but can struggle with process nonlinearity (Mears et al. 2017). More advanced techniques like Model Predictive Control (MPC) improve stability by predicting future process behavior, yet they require high computational power and an accurate process model. Artificial Intelligence (AI)-driven approaches, including fuzzy logic (Mears et al. 2017; Cheng et al. 2023), neural networks, and reinforcement learning, offer adaptive control for highly nonlinear and dynamic processes with little to no model information such as self-driving cars (Cutler and How 2016), automated game play (Silver et al. 2016), and Robotics (Guliyev and Parsayan 2022).

Reinforcement learning (RL) has recently been applied to fermentation set-point control. In brief, RL refers to training a control policy based on iterative testing of a representative environment, much as a human brain does to learn. This has been demonstrated on set-point control of temperature in bioreactors producing ethanol, amino acid feeding levels to microbial co-cultures, and glucose feed levels to yeast (Teo et al. 2010; Treloar et al. 2020a; Rajasekhar et al. 2024). In each case the RL agent is trained through a reward function based on the deviation from a known setpoint, without any other model or understanding of the physical system. There have also been examples of incorporating first-principle models and plant historical data to further simulate process dynamics for RL-based feed and temperature control (Ma et al. 2021). Additionally, most recent RL-based process control approaches, including methods that modify action update frequency to improve stability, rely on digital-twin simulations for evaluation and do not explicitly discuss deployment in real physical systems. In this work, the term "digital twin" is used in an aspirational sense to describe a simulation framework intended to represent and support future interaction with a physical system. We clarify that the present study is purely simulation-based, and no real-time physical counterpart is currently integrated. The model is therefore more appropriately interpreted as a virtual training environment designed to enable future development toward a true digital twin implementation.

While the Monod-based model provides a simplified representation of cellular growth and does not capture all full nonlinear regulatory and metabolic complexity of real biological systems, it was intentionally selected as a minimal digital twin for initial policy training. More detailed kinetic or metabolic models incorporating pathway-level regulation could potentially improve policy robustness and generalizability; however, they also introduce additional parameters that may be difficult to identify accurately, risking overfitting to model artifacts rather than true process dynamics. The purpose of the in-silico environment is not to perfectly reproduce biological reality but to provide the RL agent with a head start on learning general control behaviors, substantially reducing the number of expensive training runs required on the physical system. The agent's policy can subsequently be fine-tuned with real process data to account for dynamics not captured by the simplified model, following established sim-to-real transfer practices in RL-based control (Salvato et al. 2021). Moreover, they all assume that there is a known optimal level of some measurable parameter to which you are controlling. In the scenario of secreted enzyme production, this optimal level is unknown.

Here we describe the use of model-free RL to control a fed-batch reactor. First, we detail the digital twin (simulation) of a fed-batch fermenter based on the Monod model of cell growth and use this to train and characterize a model-free RL controller with the unconstrained objective of maximizing secreted enzyme activity in the fermentation through intelligent timing of media supplementation. Next, we describe testing of different RL agents and various reward functions to determine which can achieve best control of media addition. We then compare this RL controller to a traditional controller that has been optimized for a fixed recipe. These are benchmarked against each other in terms of experimental cost to develop, ability to account for changes in cell seeding quality, and their ability to control for process perturbations such as changes in temperature or feed addition. We then determine the experimental cost to train a new cell type on an agent that has been largely trained by the *in-silico* environment, indicating the practical applicability of this technique to actual fermenters.

2 | Methods

2.1 | Simulation Environment

An *in-silico* simulation environment (mathematical model) was built to train and evaluate the reinforcement learning agent, and benchmark against, traditional fixed models (section 4.3). A fed-batch reactor growing *Aspergillus niger* on maltose-based media to produce glucoamylase (Metwally 1997) was used to frame our environment (Figure 1). However, this specific fed-

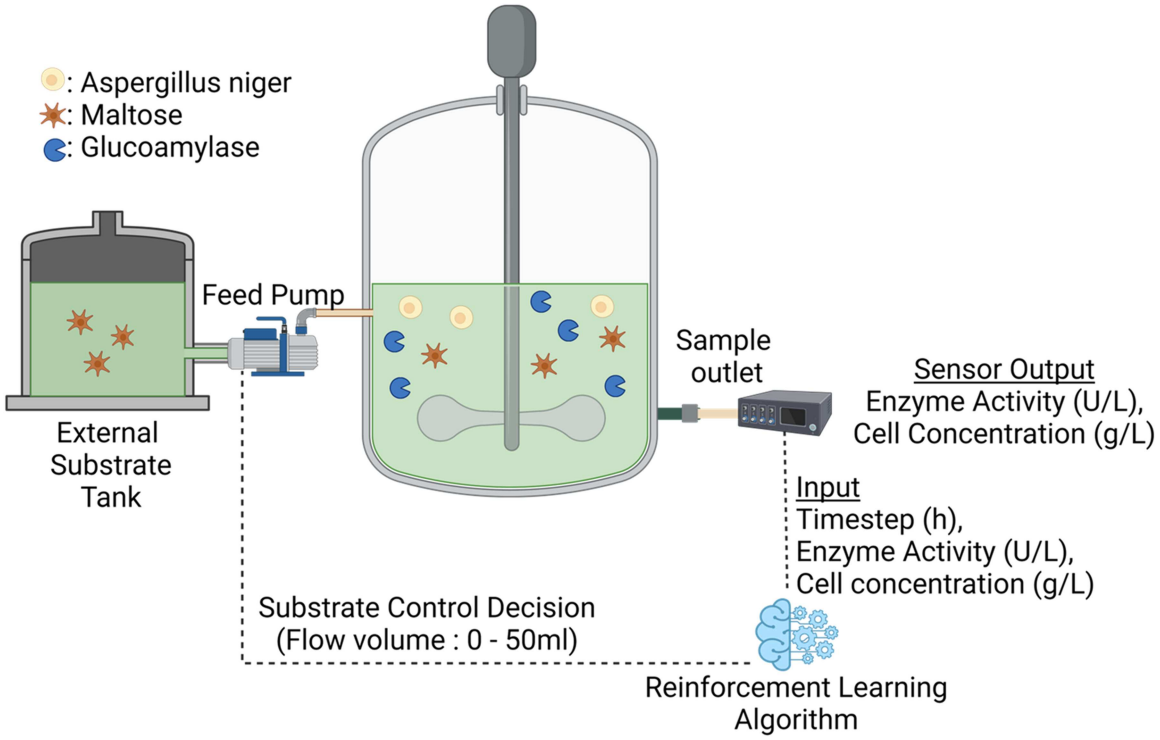


FIGURE 1 | Overview of simulation environment (mathematical model) for fed-batch production of cells expressing a secreted product, in this case *Aspergillus niger* secreting glucoamylase.

batch model could be used for many different microbial cell factories producing a secreted product by adjusting model parameters. This model tracks the changing concentrations of cells (X), general carbon substrate for growth (S), and the enzymes produced (E). The simulation features one control parameter, the substrate addition volume, which can be dynamically adjusted (0–50 mL), while the feeding frequency is fixed through predefined intervention intervals, to influence cell growth and enzyme production. The goal of the RL agent is to maximize the secreted enzyme in the media, as this would be collected, filtered and packaged as the commercial product (Arnau et al. 2020).

The environment is built from a system of three ordinary differential equations that use the first-order Monod model (Lobry et al. 1992) to describe the interplay between substrate concentration, microbial growth, and enzyme production (Equations 1–3, see example simulation run in Figure 2a). Note that the units for enzyme amount are the measured activity per unit volume of the sampled fermentation broth. This is the relevant parameter that would be measured at-line using emerging sensor modalities (Kuo and Reuel 2025) and fed to the RL agent. These equations are numerically solved using Euler's method at a simulation time step Δt of 36 s (Figure 2b). We acknowledge other numerical approaches such as Heun or Runge-Kutte can be implemented to gain computational efficiency, however at the small Δt step-size used, comparing the methods we did not observe a significant advantage for our simulation goals; thus, we proceeded with the simple Euler method. Each simulation run lasts for 24 h of simulated time unless one of the predefined early termination conditions is triggered. Early termination occurs if (1) the fermentation tank reaches its maximum working volume of 1.5 L, or (2) substrate limitation causes cell growth to cease (i.e, $\mu_{\max}$ drops to zero due to insufficient substrate concentration; Figure 2c). In addition, certain constraints are intentionally imposed to restrict exploration to realistic operating regions; for example, the simulation is terminated early if the system approaches substrate inhibition. This helps the RL algorithm learn to maintain the substrate within a specific range, rather than wasting time exploring in obviously less advantageous regions. The parameters for these coupled equations (Equations 1–3) are described in Table 1.

$$\frac{dX}{dt} = \mu_{\max} \frac{S_t}{K_s + S_t} X_t \tag{1}$$

$$\frac{dS}{dt} = -\frac{1}{Y_{XS}} \frac{dX}{dt} \tag{2}$$

$$\frac{dE}{dt} = \mu_{e_opt} f(\sigma_t) X_t \tag{3}$$

$$f(\sigma_t) = \frac{k}{\lambda} \left(\frac{\sigma_t}{\lambda} \right)^{k-1} e^{-\left(\frac{\sigma_t}{\lambda}\right)^k} \tag{4}$$

$$\lambda = \frac{\sigma_{opt}}{\left(\frac{k-1}{k} \right)^{\frac{1}{k}}} \tag{5}$$

In Equation 1, $\frac{S_t}{K_s + S_t}$ determines the cell production rate on the basis of current tank substrate concentration S_t. This accounts for the physical effects of low substrate which is shown to reduce the growth rate of cells and the equation also makes sure that the excess substrate does not exceed $\mu_{\max}$. It is also shown in Figure 2c, via the bar overlay, the extent to which the max

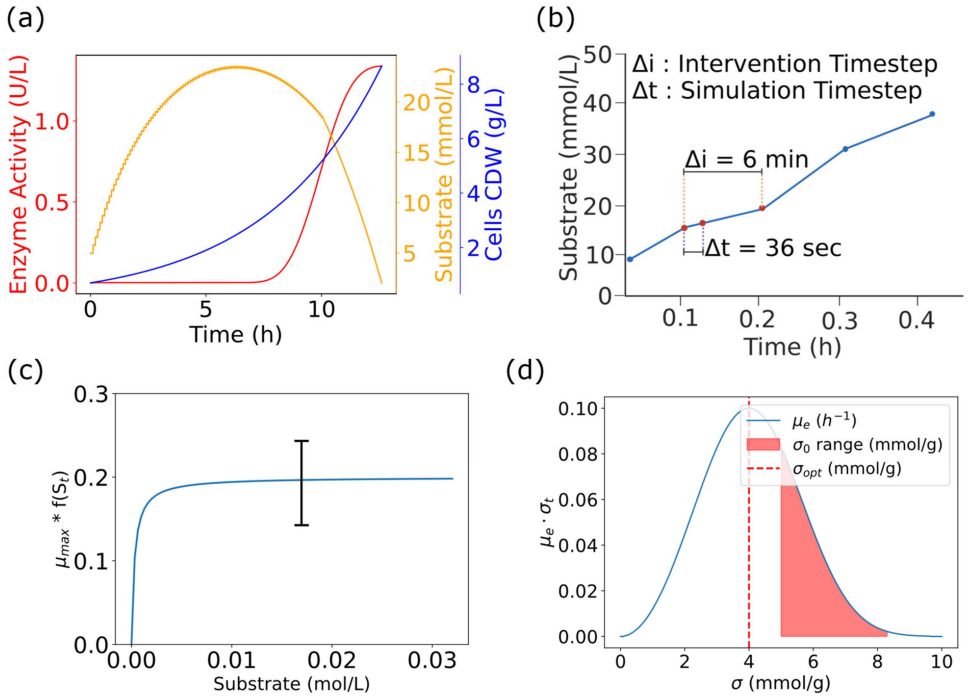


FIGURE 2 | Fed-batch simulation output and modifications in terms of cell production and enzyme production (a) Example numerical solution of the Monod model used in the training environment tracking the change of cell, substrate, and enzyme amount as a function of time when a constant volume (mL) of substrate is added every 6 min (Δi). (b) Simulation time intervals: Δt represents the timestep used in the Euler method and Δi indicates the interval for substrate addition to the tank. (c) Effect of substrate concentration S_t on the cell growth rate modeled using Monod's equation. The variation in $\mu_{\max}$ used during training is shown with the error bar. (d) Effect of the σ_t on enzyme production rate. The initial cell concentration X_0 varies during training, altering the σ_0, as depicted by the shaded region in the figure.

TABLE 1 | Parameters used in the fed-batch model used as the RL test environment.

Symbol	Description	Initial value	Units
X_t	Microbial cell concentration at time t	0.6–1.0 (Metwally 1997)	g/L
S_t	Substrate concentration at time t	5 (Metwally 1997)	mol/L
K_s	Half saturation constant	0.0003 (Metwally 1997)	mol/L
E_t	Enzyme activity at time t	0.0	U/L where U is an arbitrary relative unit of activity (dependent on assay used) (Bisswanger 2014)
$\mu_{\max}$	Maximum specific growth rate	0.15–0.25 (Metwally 1997)	h^{-1}
Y_{XS}	Yield coefficient for cell growth	210 (Metwally 1997)	g cells/mol substrate
μ_{e_opt}	Maximum specific enzyme production rate	0.1 *[value chosen to align with industry-standard enzyme activity levels.]*	h^{-1}
σ_0	Initial substrate to cell ratio	5–8.3 [Calculated using X_0 & S_t]	mmol/g
	Substrate to cell ratio optimum	4 *[Value set to provide realistic levels of cell growth in simulation]*	σ_{opt}
$f(\sigma_t)$	Weibull function describing relative enzyme production based on substrate to cell ratio	Range is 0 to 1	Unitless
λ	Scale parameter in Weibull function	~0.0045	Unitless
k	Shape parameter in Weibull function, controls the curve's spread	3	Unitless

growth rate is stochastically adjusted in our learning simulations. This allows the agent to train cells with a range of varied growth rates, allowing it to learn how to adjust substrate feed rate (at the substrate addition time step Δi of 6 min, see Figure 2b) to account for this change and stay within the optimal range for cell growth to occur.

In Equation 3, $f(\sigma_t)$ attenuates the maximum rate of enzyme production (μ_{e_opt}) on the basis of current substrate-to-cell ratio (σ_t) in the tank, which follows a Weibull distribution with its peak indicating optimal conditions for enzyme yield (Figure 2d, dotted line at σ_{opt}). This approximates the physical phenomenon of cell excess adaptation, conserving metabolic resources in low substrate situations and turning to alternate product pathways when the substrate is present in excess (Dantas et al. 2021). In the training runs, the initial cell concentration (X_0) is varied, presenting a range of initial reactor seed densities, which corresponds to a range of initial σ_0 (shaded region in Figure 2d). Note that the range is intentionally above the optimal ratio as the cells will always be seeded at a low density compared to the high initial feed substrate present in the tank. This means that for every training run, an optimal agent would wait to add the initial bolus of carbon substrate and then decide how to add more over time to maintain the optimal $\sigma_t$in the tank to promote maximal enzyme production. For the functional form of $f(\sigma_t)$ we chose the Weibull model because it is non-symmetric and ensures that it tapers smoothly to zero at a $\sigma_t$equal to zero regardless of the optimal point.

The feeding timestep of 0.1 h (6 min) was selected through empirical evaluation of five candidate intervals (0.01 h, 0.05 h, 0.1 h, 0.5 h, and 0.9 h), each used to train a separate SAC agent under identical conditions. The 0.1 h interval achieved the lowest mean SCR RMSE and was consistent with the characteristic substrate consumption timescale of the system. Shorter intervals provided observations too similar between steps for effective reward differentiation, while longer intervals were insufficient for timely corrective action. The full sensitivity analysis is presented in Supplement 9.

2.2 | Reinforcement Learning Agent

At each 'intervention' timestep (Δi, see Figure 2b), the reinforcement learning agent observes the current state of the environment (enzyme activity level (E_t), cell concentration (X_t), and overall run time (t)), selects an action according to its updated policy (a strategy for action selection), and executes the chosen action (volume of substrate to add ranging from 0 to 50 mL, Figure 3a). This results in a transition to a new state, and the generation of a reward signal numerical feedback indicating the effectiveness of the action taken. The reward function plays a critical role in shaping the agent's learning process. The agent is rewarded when the change in enzyme activity ($\Delta E > 0$) is positive, receiving a reward of +10. If there is no change in enzyme activity ($\Delta E = 0$), a penalty of -10 is applied. To further encourage continuous improvement, a significant bonus reward of +100 is granted when the increase in enzyme activity surpasses the previous positive change (Figure 3a). Figure S10 from supplementary shows how a learned agent is trying to achieve a + 110 every Δi. Unlike some traditional RL setups (Treloar et al. 2020b), no fixed goal reward is defined, as the maximum achievable enzyme activity is not predetermined. Instead, the agent strives to achieve the highest possible enzyme production in its exploration.

The choice of a discrete reward structure over a continuous formulation was motivated by the dynamics of the fermentation process. Enzyme activity increases are naturally small during early stages and substantially larger near the end as enzyme levels accumulate. Continuous reward functions proportional to ΔE caused the agent to disproportionately prioritize later stages of the fermentation, resulting in slow and unstable learning of critical early feeding decisions. A discrete reward structure assigning a fixed positive reward for any improvement in enzyme activity and a negative reward for no change provides consistent reinforcement across the entire fermentation trajectory regardless of the absolute magnitude of ΔE. This formulation led to faster and more stable policy convergence. Several reward formulations were evaluated, including a continuous function scaled between 0 and 10 based on enzyme produced per intervention timestep, a relative change formulation based on ΔE, and a discrete structure with a bonus reward when the current enzyme change exceeds the previous change. A comparison of these reward functions is provided in Figures S1–S3 in the supporting material.

The main goal of the RL agent is to optimize the feeding strategy to maximize enzyme production. It achieves this by learning to continuously increase enzyme production by maintaining a substrate concentration that can achieve an increased rate of cell production which indirectly and maintains σ_t at σ_{opt}, all without prior knowledge of the ideal ratio (σ_{opt}). By treating the simulation as a black box, the agent must deduce the best strategy through training, driven by the reward function. As shown in Figure 3a, the reward function does not explicitly define the σ_{opt}. Instead, it incentivizes the RL agent to maximize enzyme production, thereby indirectly maintaining the optimal ratio.

Gymnasium, an open-source Python library, was used to code the custom fed-batch fermentation simulation environment. Stable Baselines3, an OpenAI toolkit, provided the RL algorithms used for training within this environment. The RL agent was trained for up to 10,000 simulation episodes (fermentation experiments), with each episode introducing different initial conditions for cell concentration X_0 and the maximum cell growth rate $\mu_{\max}$ (Figure 3b). This variability exposed the agent to diverse conditions, allowing it to adapt to dynamic system behaviors. Training performance was periodically evaluated every 150 training episodes, ensuring the agent had enough time to update and explore new strategies. During evaluation, training was paused, and the RL agent with its current knowledge is assigned to control feed addition in a separate simulation environment where initial conditions remained fixed for fair comparison between each test. Each evaluation phase consisted of 50 episodes, where the enzyme activity from each episode was recorded, and an average was calculated across all episodes. The model achieving the highest overall average value during the testing phase is designated as the best model, and its weights are saved accordingly. Training was considered converged if this average did not improve across 30 consecutive evaluations, indicating that the RL agent had found the best possible policy. Even though the simulation

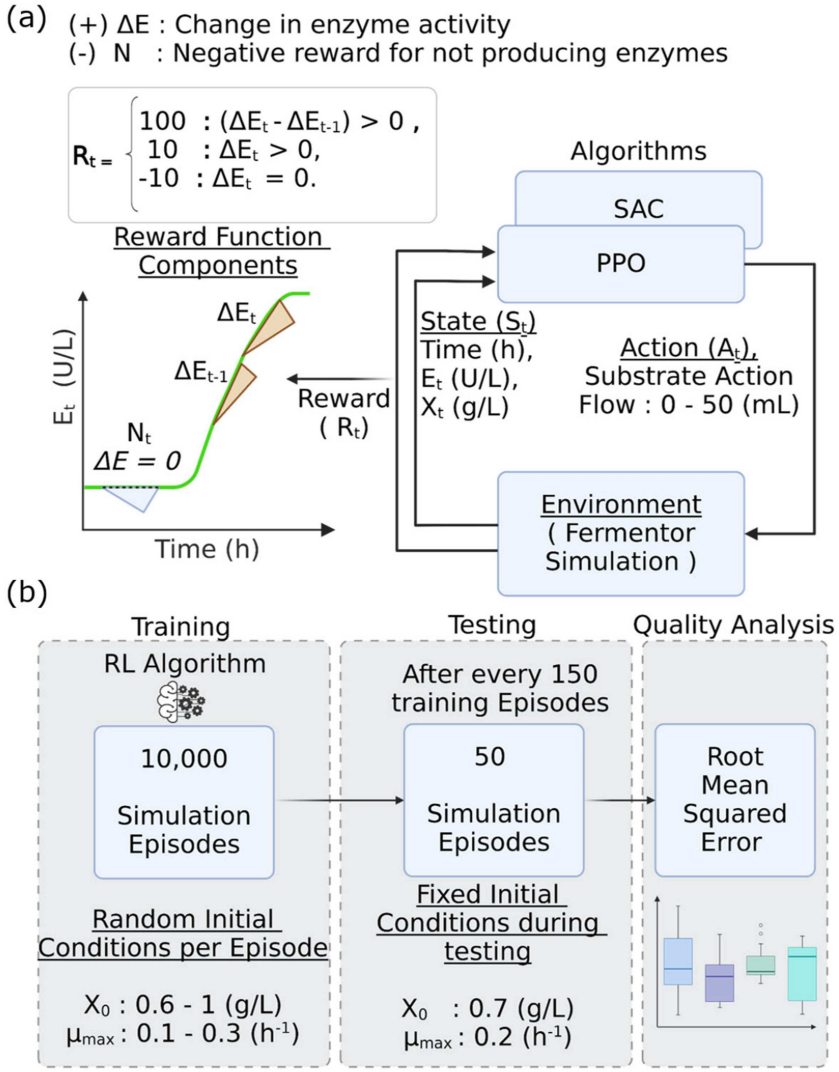


FIGURE 3 | Reinforcement Learning (RL) cycle and training methods. (a) The iterative RL cycle illustrates the core components including agent algorithms used for training, the defined action and observation states from the environment, and the design and weighting of the reward function that guides the agent's learning. (b) An outline of the sequential stages to evaluate performance of an RL agent including training, testing, and quality measurement.

environment is deterministic, the RL model uses a stochastic policy for prediction, causing results to vary for each test run. This encourages the RL agent to exhibit dynamic and adaptive behavior.

To evaluate the effect of observation uncertainty on controller performance, Gaussian noise of varying magnitude was injected into the state observations during evaluation. As shown in Figure S11, the baseline controller trained on clean observations achieves the lowest RMSE. The relationship between noise level and performance is non-monotonic: moderate noise ($\sigma = 0.001$) degrades performance most severely, while higher noise levels ($\sigma = 0.05$) show partial recovery. This is consistent with prior work showing that small observation perturbations can destabilize precision-trained RL policies, while larger perturbations encourage more conservative strategies (Zhang et al. 2020). These findings highlight the importance of accounting for sensor noise during training when deploying RL-based controllers with emerging sensor modalities.

Since reinforcement learning does not provide formal guarantees of global optimality for nonlinear control problems, convergence in this work was defined using a practical stopping criterion based on evaluation performance. To reduce the likelihood of convergence to poor local optima, stochastic policy exploration, randomized episode initializations, and periodic evaluations across multiple rollouts were employed during training. These strategies provide empirical confidence that the learned policy represents a robust high-performing solution, although global optimality cannot be strictly guaranteed.

During the analysis phase, the best model saved from the testing phase is used to run 100 simulations with fixed initial conditions. The agent's ability to regulate σ_t is evaluated by measuring the deviation from the optimal level σ_{opt} at each time point Δt and calculating the root mean squared error (RMSE). A lower RMSE indicates more precise control.

Two model-free RL algorithms were evaluated in this work: Proximal Policy Optimization (PPO) and Soft Actor-Critic

(SAC). Each algorithm takes a distinct approach to policy optimization and exploration. PPO is an on-policy algorithm, meaning it learns from data generated by its current policy and updates it gradually to balance simplicity and performance (Schulman et al. 2017). It explores different actions, receives feedback based on rewards, and refines its strategy step by step for stable learning. In contrast, SAC is an off-policy reinforcement learning algorithm, which learns not only from current actions but also from past experiences stored in memory. It consists of an actor, which selects actions, and two critics, which assesses their effectiveness. To enhance learning, SAC introduces randomness in decision-making, encouraging exploration and preventing the agent from settling too quickly on a suboptimal approach (Haarnoja et al. 2018). These algorithms are benchmarked to determine which approach was more effective in optimizing enzyme production based on their different learning strategies. The primary criterion for selecting the best model is RMSE analysis, which provides insight into how effectively $\sigma_t$is maintained at its optimal value σ_{opt} (Figure 3b).

3 | Results and Discussion

3.1 | Reinforcement Learning Training and Performance

Both the PPO and SAC algorithms, when trained using the framework in Figure 3b, were able to control feed additions to maintain the σ_{opt}. Figure 4a visually represents how we calculate RMSE to get an idea of how close σ_t values are with respect to σ_{opt}. 1Our first objective was to determine effective reward functions, settling on the one detailed in Figure 3a where rapidly increasing enzyme production is heavily rewarded, and no growth is penalized (Supporting 1, Figures S1, S2, and S3). Between PPO and SAC there was varying performance with regards to maintaining the (σ_{opt}) as shown by RMSE from the optimum level (Figure 4b), with SAC exhibiting tighter control and less variation over the 100 test runs. By examining the learning curve (Figure 4c) where the average episode reward is tracked as the number of episodes progresses, it is apparent that SAC outperforms PPO by achieving a higher reward while maintaining stability and adhering closely to the optimal policy. PPO demonstrates "catastrophic forgetting" (Zhang et al. 2021) where the model abruptly and significantly loses previously learned information as it acquires new knowledge during training. SAC's soft policy updates allow for gradual adjustments, promoting exploration without drastic changes, whereas PPO's larger exploration jumps can help find the global maximum but may also introduce instability by oscillating between optimal and suboptimal regions. The individual episode traces are also plotted for the training runs (Figure 4d, SAC shown) and tight performance is observed. SAC has 62.86% lower RMSE compared to PPO and was selected for all following tests.

Compared to PPO, SAC demonstrated more stable learning under the proposed reward formulation, which provides discrete and threshold-based feedback ($+10/-10$ rewards with occasional large bonus signals). Such reward structures introduce high variance and non-smooth gradients, making policy-gradient updates in PPO sensitive to sampling noise and less sample-efficient. In contrast, SAC leverages off-policy learning with a replay buffer, allowing rare high-reward experiences to be reused multiple times, thereby improving credit assignment across delayed system dynamics. Additionally, entropy regularization in SAC promotes sustained exploration, enabling the

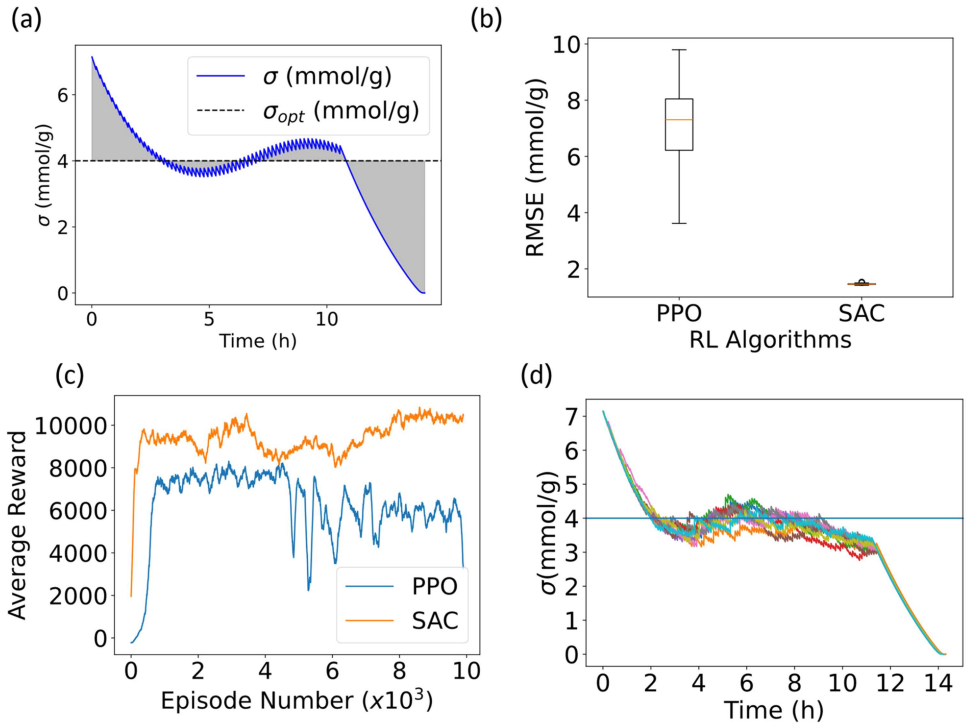


FIGURE 4 | RMSE Evaluation and Optimal RL Control. (a) RMSE computation to assess the effectiveness of the substrate feeding strategy by measuring its deviation from the optimal ratio. (b) RMSE comparison of trained RL agents to identify the best-performing model. (c) Training progress and learning curve of the RL algorithm. (d) σ_t regulation achieved by the top-performing RL agent.

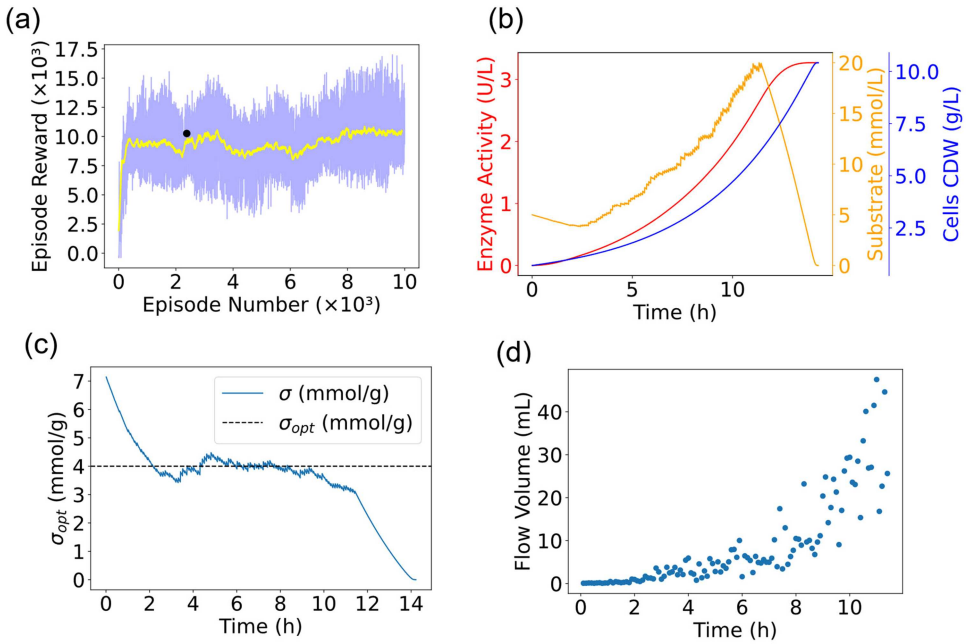


FIGURE 5 | Example test run from 95% trained SAC agent. (a) Improvement in RL training as function of episodes for SAC algorithm indicating training point for 95% quality (marked by a dot) from which is shown control data of: (b) substrate, cell, and enzyme levels (c) (σ) compared to optimal set point, and (d) feed volume additions.

agent to more reliably discover control actions that lead to large positive enzyme activity improvements. These properties make SAC better suited for the sparse, discontinuous, and delayed reward landscape of the fed-batch fermentation control problem.

Next, we directly assess the control performance of the SAC agent trained to 95% maximum quality. Evaluating the 10,000 training episodes of the SAC agent (Figure 5a), it is observed that 77.7% of the training occurs within the first 200 episodes and that 95% performance is achieved at 2400th episode (indicated by solid point in 5a) compared to PPO which reaches only 80% reward in 2912 episodes (Supporting 5, Figure S8)). To evaluate if this is sufficient training to control the fed batch environment, the agent trained to this point (solid point in Figure 5a) is used on a test run and the enzyme, substrate, and cell levels are observed, the σ_t is compared to optimum, and the feed media volume additions are assessed (Figure 5b–d). The trained model has found the σ_{opt} and can control growth substrate additions to account for the increased cell growth. The simulation begins with a low cell concentration and a high substrate concentration, allowing the existing substrate to support cell growth. As shown in Figure 5d, the RL agent initially supplies a minimal amount of substrate to enable cell growth and bring the σ_t to its optimal value. Once this ratio is achieved, the agent increases substrate in addition to maintain balance. As cell concentration increases, the RL agent continuously adjusts substrate input to match the growing demand. Since the reward function does not define a fixed endpoint reward, the agent must occasionally deviate from the optimal ratio to explore whether adding or withholding substrate results in a higher reward. Towards the end of the experiment, when the tank reaches full capacity, substrate addition stops, and the remaining substrate is consumed by the cells. This explains the decline in substrate concentration observed in Figure 5b,c.

3.2 | Traditional Controller

To test the utility of the RL control approach, we next benchmarked against a traditional control strategy. In this case, there is no known set-point, but rather a maximization objective to the controller. In this case a set of experiments could be run for a new production cell and enzyme product, in which an optimal strategy for feed rate production is set. This would be experimentally much less expensive than the RL approach but would have the disadvantage of being a static recipe that could not readily account for process changes. To test this, we developed a traditional model that is tuned based on (1) initial feed volume (V_o), (2) feed ramp factor r_f(rate at which the feed volume increases), and (3) feed addition interval Δ_f (time between each substrate addition Δi). We used both Design of Experiments (DoE) and a hyperparameter optimization framework called 'Optuna' (Akiba et al. 2019) that employs a Bayesian optimization (BayesOpt) algorithm (Tree-structured Parzen Estimator) to search a reasonable parameter space for the best parameters of substrate addition that maximizes the enzyme activity level (example setting levels and flow addition profile shown in Figure 6a). Three different DoE designs (Box-Behnken Design (BBD), Central Composite Design (CCD), and Full Factorial Design (FFD)) with 15, 22, and 125 experimental runs respectively are used to show the positive effect of more exploration (Figure 6b, orange bars). The BayesOpt algorithm explores its search space adaptively, generating different optimal substrate addition strategies in each run (stochastic). To account for this variability, we ran BayesOpt 20 times for each run number category (15, 22, 125), allowing it to generate a unique strategy in each iteration. We then selected the strategy that achieved

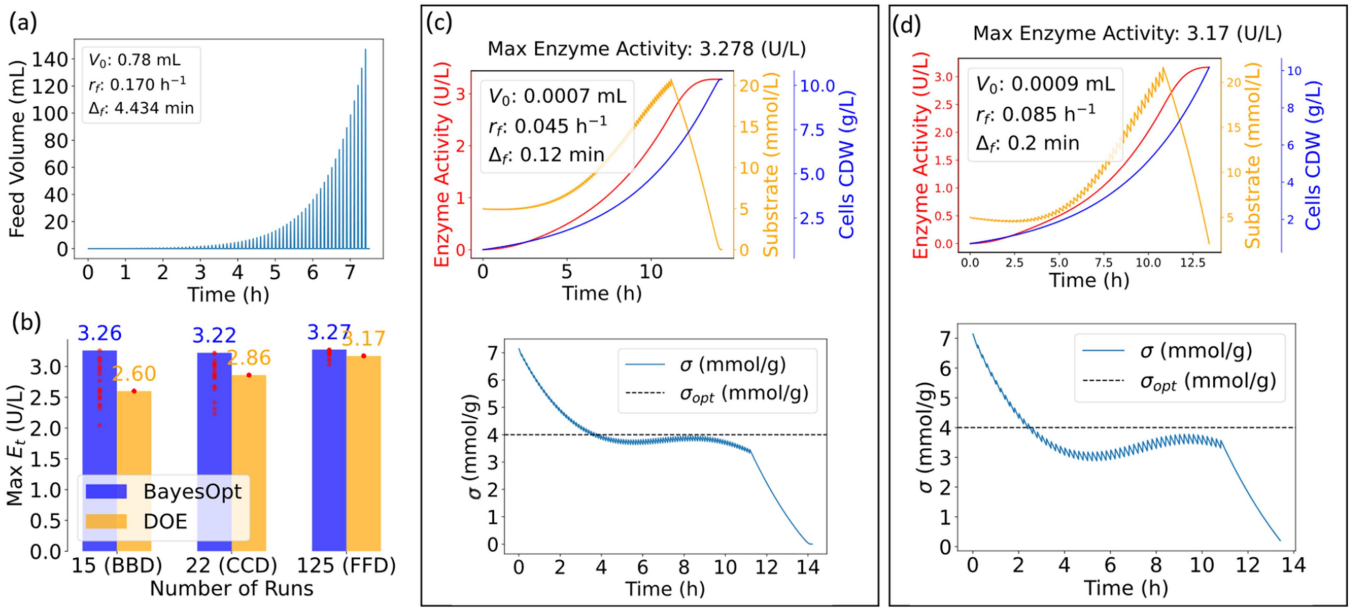


FIGURE 6 | Tuning a Traditional Controller with Bayesian Optimization and Design of Experiments. (a) Parameters used to control substrate flow addition in the traditional model and example flow addition profile. (b) Comparing Bayesian optimization and Design of Experiments (DoE) in tuning optimal parameters for the feed model, shown at increasing experiment runs. Bayesian optimization is stochastic, showing range of outcomes with red dots. DoE methods include Box-Behnken Design (BBD), Central Composite Design (CCD), and Full Factorial Design (FFD). Example dynamic responses and resulting enzyme activity levels for (c) Bayesian and (d) Design of Experiments (DoE) optimized feed model.

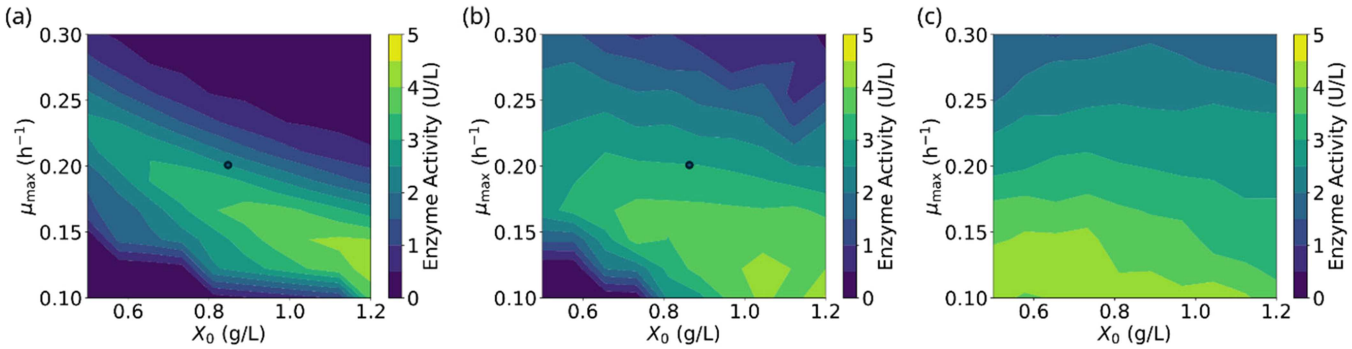


FIGURE 7 | Comparison of RL and traditional controllers under varying initial cell concentrations (X_0) and maximum cell growth rates (μ_max), with enzyme production levels observed across the test set. The centre dot (•) in (a) and (b) marks the fixed X_0 value used during training. (a) Traditional controller with fixed substrate addition recipe. (b) RL agent trained with fixed starting conditions. (c) RL agent trained with starting conditions spanning the same range as the test set.

the highest enzyme activity and presented it in Figure 6b (blue bars). BayesOpt conducted using Optuna with the same experimental run numbers shows that this approach has more variability at low experimental runs but performs similar to DoE. Inspecting individual runs controlled by DoE and the parameter optimization tool (Figure 6c-d) shows very similar process control profiles. In benchmarking against RL, we use the BayesOpt approach to tune the optimal traditional controller.

3.3 | Benchmarking Reinforcement Learning vs Traditional Controller

With the traditional controller established, we next compare performance of the two approaches to 1) varying levels of cell growth and Initial Cell concentrations observed in fermentation processes and 2) potential system perturbations. For the first subject, cells can vary in their expansion rate based on variations in the viable cell seeding density and cold storage conditions. To test this effect, we exposed the trained agent and traditional model to a range of starting cell concentrations (X_0) and maximum cell growth rates (μ_{max}) and determined what level of enzyme productivity that could be obtained (Figure 7). For the traditional controller, there is no model adaptation, and therefore the resulting enzyme production is isolated to a narrow band of cell parameters that align with the proscribed substrate additions (Figure 7a). The RL model trained with fixed initial conditions is able to adapt to unseen scenarios (Figure 7b) better than traditional controller is because of the explorative nature of training RL models where they understand the underlying dynamics of the system through interactions and are able to adapt. When the RL is trained with varying start conditions, the resulting control is much improved,

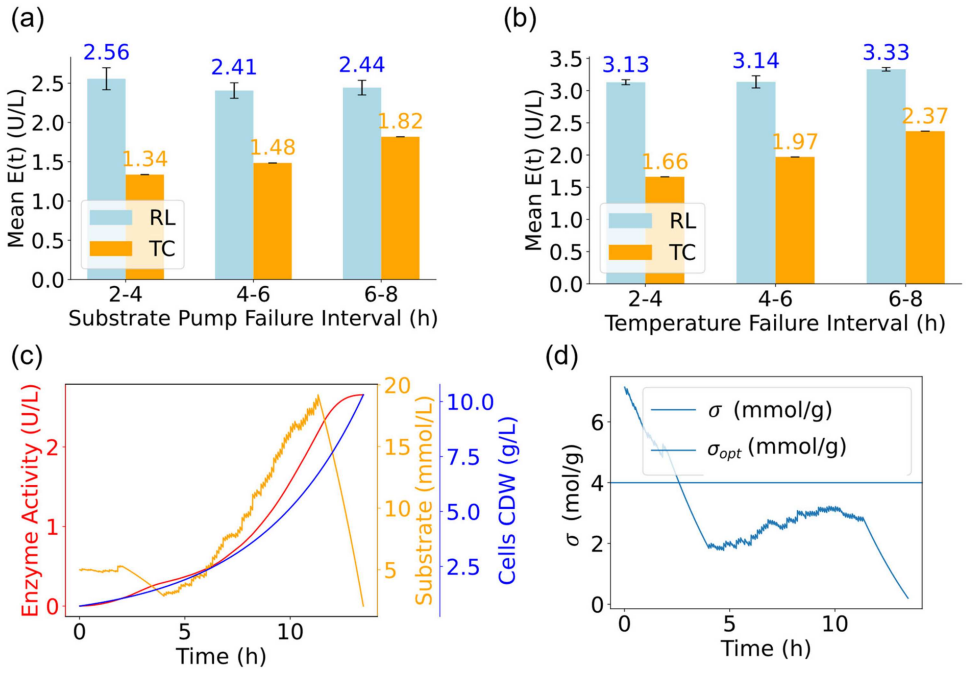


FIGURE 8 | Effects of process perturbations on RL and traditional control performance. (a) Comparison of RL and the traditional controller during a substrate pump failure; time periods of failure noted on the x-axis. (b) Comparison of RL and the traditional controller during a temperature jacket failure; time periods of failure again noted. (c) Growth parameter trends during the substrate pump failure between the 2nd and 4th hour for RL controlled strategy. (d) σ_t regulation during the substrate pump failure for RL controlled strategy.

enabling maximum level of enzyme production at all conditions (Figure 7c). Note, that the banded levels of enzyme activity are now resulting from the physically obtainable levels for the varying amounts of cell production.

It is important to note that the comparison between ML-based and traditional hard-coded controllers is inherently unequal, as ML models learn and adapt to unseen data, whereas hard-coded controllers follow predefined rules. The justification for this comparison lies in the fact that no two fermentation runs whether in industry or small-scale settings are ever identical due to variations in initial conditions and external or internal factors. While hard-coded approaches may require fewer experiments to find an optimal path, they are likely to underperform when unforeseen factors arise, limiting their ability to achieve maximal enzyme activity as effectively as RL.

Next two, real-world process perturbations were used to explore performance of the RL control process versus the traditional controller in adapting to unforeseen process changes (Figure 8). The first scenario models when the substrate flow pump fails, stopping substrate addition to the tank. With no new substrate added, cells will consume the remaining substrate, causing the σ_t to deviate from the optimal point (Figure 8c). The time points of failure were staggered (between 2 and 4, 4 to 6, and 6 to 8 h into the culture) and it was found in all occurrences the RL control method could better adapt and preserve a high level of resulting product. The second perturbation models a failure in the temperature control, leading to a rise in tank temperature, which slows down cell growth (modeled as μ_{max} peaking at an optimal temperature and then decreasing away, using an Arrhenius model, see Supplementary 3). We assume that before temperature failure occurs the reactor temperature is at the optimum and due to failure it rises slowly, reducing the cell production rate; once the issue is fixed, the temperature goes back to the optimum (Supporting 3 Figure S6b). It was found that RL control maintained high enzyme production whereas the traditional control decreased (Table 2). It is important to note that the RL agent has not been specifically trained for these perturbations but can take the appropriate action when the σ_t moves away from the optimal position, based on its prior, exploratory training.

TABLE 2 | Performance improvement of RL over traditional controller under process perturbations.

Substrate pump failure		Temperature failure	
Hours	**% Increase**	**Hours**	**% Increase**
2–4	91.04	2–4	88.55
4–6	62.83	4–6	59.39
6–8	34.07	6–8	40.51

3.4 | Adapting an In Silico Trained RL Model to a New Cell Type

An RL agent trained in a *silico* fed batch fermentation environment for tens of thousands of experiments is capable of robust control. However, many of the parameters used in this environment were derived from a single cell type (Metwally 1997) or assumed based on expected growth profiles. To make use of this approach in an actual context, the *in silico* trained RL agent would need to be updated with actual fed batch experiments. However, it is not clear if this updated testing would require as many tests ($n = 10{,}000+$). To explore the cost of updating the agent to a new cell type, we

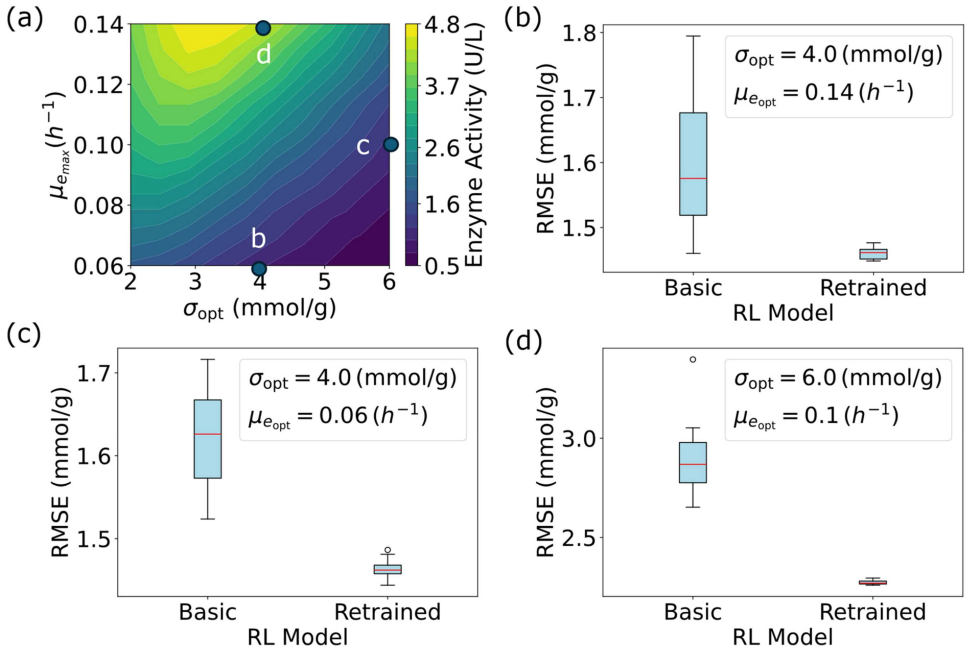


FIGURE 9 | RL model adapting to a new cell type. (a) RL agent trained with single cell type at different seed conditions tested against a range of new cell types, modeled through changing σ_{opt} and μ_{e_max}. (b), (c), and (d) highlight corner cases where retraining the model enhances its ability to optimize σ_t control.

explore this with our current model. To approximate a new cell type, we change the optimal $\sigma_{opt}$and the maximum enzyme production rate μ_{e_opt}, indicating a cell with a new enzyme productivity and metabolic burden. Surprisingly, an RL model trained on a single cell type with fixed $\sigma_{opt}$and μ_{e_opt} with varying initial conditions (the agent used above in Figure 7c) is capable of robust control across the design space of the new cell type (Figure 9a) with no additional training. This augurs the use of an *in silico* trained RL agent in a physical environment if the digital twin assumptions are reasonably close to the physical environment.

Similar to the training and testing described in section 2.2, we update the pre-trained SAC model with additional episodes on new cell types and use the same criteria for convergence (30 consecutive evaluations remaining below the maximum evaluation achieved). We monitored how many episodes it took for the previous model to adapt to the new cell type parameters (see Supplementary 2). After retraining the SAC model on three edge cases, we found that it took an average of 3,972 episodes for the RL model to fully converge to a new cell type. The total reward does not exhibit as much of a change as is observed from training from scratch (Supporting 2, Figure S5), however there is an improvement in the model's consistency as shown by a lower and tighter error for most cases (Figure 9b–d). Retraining each case individually resulted in a 13.29% improvement for the first corner case, 19.23% for the second, and 0.3% for the third (which was already near its optimal value). Notably, the model reached 95% of the maximum training achieved within the first 100 training experiments, indicating that not many physical experiments would be needed to reasonably augment the agent. This is likely dependent on how closely the computational system models the behavior of a real physical system; however, it indicates that the model does not need to be a perfect match to the target physical system. Such comparison will be the subject of future work as we connect the RL agent to physical systems.

Looking beyond these edge cases, we assessed the effectiveness of updating the training of the agent ("retraining") versus forgetting the prior policy and training on the new environment from scratch for ten $\sigma_{opt}$and ten μ_{e_max} combinations spanning the design space shown in Figure 9a. For these 100 test cases, we used the same framework detailed in Figure 3, namely assessing the agent performance (using RMSE from the σ_{opt}) on 10 test cases after every 150 episodes of training. These periodic RMSE measures provide insight into the overall trajectory of the error curve, indicating whether it is trending upward or downward. Ideally, the RMSE should decrease over time as the model becomes more specialized in the new environment. However, in some cases, exploratory behavior during training may lead the RL agent away from the optimal policy, resulting in temporary increases in RMSE (Supporting 6, Figure S9c–d). Additionally, we also monitor the cumulative reward obtained in each episode. For every experimental condition, we compute the average cumulative reward across all episodes to assess the overall performance of the agent at that specific parameter setting (Supporting 6, Figure S10).

From this analysis a few things are apparent about adapting the RL model to a new cell type: 1) it is largely insensitive to changes in enzyme productivity (μ_{e_max}); a cell can make more or less target product and it can still adapt to control production with little need for updating the policy. 2) The model requires additional training for cells that have a different optimal σ_t for enzyme production (different metabolic burden and ideal substrate window). Of special interest, it is observed that for new cells that require less substrate than what the agent was trained on, it improves on retraining, whereas for new cells that require more substrate the agent is better off forgetting the prior policy

and retraining. This makes intuitive sense, as the trained agent has largely learned on a region where there is excess substrate (from the start of the culture) and generally learns to wait before giving small additions of substrate to augment new cell growth. In the case of a hungrier cell type, it needs a new policy that is much more aggressive in substrate additions.

4 | Conclusion

This study demonstrates that RL agents can be trained to effectively determine optimal feeding strategies that align with the underlying conditions necessary for cell growth and enzyme production. Of the two algorithms tested, the SAC agent showed superior stability, and a faster learning rate compared to the PPO agent. The SAC controller was able to achieve high levels of performance, reaching 87.4% of the maximum reward in just 190 training episodes, and 95% performance at the 2380th episode. In comparison to traditional controllers, which rely on fixed recipes optimized for specific cell types, the RL controller was shown to be more adaptable to changes in initial cell growth and process perturbations. Its ability to continuously optimize feeding strategies makes it well-suited for production cases where conditions may vary or evolve. Of note, we found the RL agent trained on one cell type can control new cell types (simulated by changes to their enzyme production levels and optimal substrate needs) without any retraining. However, the agent control tolerance (level of error) is improved with additional training. This indicates that the agent trained with the *in silico* environment could be feasibly updated with minimal testing on a physical system, especially if it is an array of reactors allowing for efficient data capture. Additionally, to acquire the state variables provided in this planned environment, sensors would have to provide periodic measure of cell quantity and enzyme activity in the fermentation broth. The first could be done with a turbidity flow cell. The enzyme measurement is more difficult to collect at the time scale necessary and in the complex media, however this is currently underway in our group using fluorescent nanoparticles (Kuo and Reuel 2025; Kuo et al. 2023; Kallmyer et al. 2018). This work focuses on the production of enzymes, but the same framework could be used to optimize the production of some other target product, if it can be measured and reported to the agent at each intervention step.

Acknowledgments

This publication was made possible with the support of The Bioindustrial Manufacturing and Design Ecosystem (BioMADE) and Schmidt Futures under award #T-SF01-A-09-0034; the content expressed herein is that of the authors and does not necessarily reflect the views of BioMADE or Schmidt Futures. We also acknowledge David Nathan and Andrew Eichenbaum (Biomade) for their feedback and suggestions for project improvements. Additionally, we appreciate the insights of Baskar Ganapathysubramanian (ISU) on RL. This work was also supported, in part, by the NIH NIGMS – MIRA ESI R35GM138265 award to Reuel.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama. 2019. "Optuna: A Next-Generation Hyperparameter Optimization Framework." *arXiv* 1: 10902. https://doi.org/10.48550/ARXIV.1907.10902.

Arnau, J., D. Yaver, and C. M. Hjort. 2020. "Strategies and Challenges for the Development of Industrial Enzymes Using Fungal Cell Factories." In *Grand Challenges in Fungal Biotechnology*, edited by H. Nevalainen, 179–210. Cham: Grand Challenges in Biology and Biotechnology; Springer International Publishing. https://doi.org/10.1007/978-3-030-29541-7_7.

Behl, M., S. Thakar, H. Ghai, D. Sakhuja, and A. K. Bhatt. 2023. "Fundamentals of Fermentation Technology." In *Basic Biotechniques for Bioprocess and Bioentrepreneurship*, 313–328. Elsevier. https://doi.org/10.1016/B978-0-12-816109-8.00021-0.

Bisswanger, H. 2014. "Enzyme Assays." *Perspectives in Science* 1, no. 1–6: 41–55. https://doi.org/10.1016/j.pisc.2014.02.005.

Chai, W. Y., K. T. K. Teo, M. K. Tan, and H. J. Tham. 2022. "Fermentation Process Control and Optimization." *Chemical Engineering & Technology* 45, no. 10: 1731–1747. https://doi.org/10.1002/ceat.202200029.

Cheng, Y., X. Bi, Y. Xu, et al. 2023. "Artificial Intelligence Technologies in Bioprocess: Opportunities and Challenges." *Bioresource Technology* 369: 128451. https://doi.org/10.1016/j.biortech.2022.128451.

Cutler, M., and J. P. How. 2016. "Autonomous Drifting Using Simulation-Aided Reinforcement Learning." In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 5442–5448. Stockholm, Sweden: IEEE. https://doi.org/10.1109/ICRA.2016.7487756.

Dantas, L. A., P. S. A. Faria, B. M. M. Dário, et al. 2021. "The Impact of Carbon Source on Cell Growth and the Production of Bioactive Compounds in Cell Suspensions of Hancornia Speciosa Gomes." *Scientific Reports* 11, no. 1: 24315. https://doi.org/10.1038/s41598-021-03845-0.

Foegeding, E. A., and M. A. Drake. 2007. "Invited Review: Sensory and Mechanical Properties of Cheese Texture." *Journal of Dairy Science* 90, no. 4: 1611–1624. https://doi.org/10.3168/jds.2006-703.

Guliyev, Z., and A. Parsayan. 2022. "Reinforcement Learning Based Robot Control." In *2022 IEEE 16th International Conference on Application of Information and Communication Technologies (AICT)*, 1–6. Washington DC, DC, USA: IEEE. https://doi.org/10.1109/AICT55583.2022.10013595.

Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning With a Stochastic Actor." *arXiv* 1: 1290. https://doi.org/10.48550/ARXIV.1801.01290.

Kallmyer, N. E., J. Musielewicz, J. Sutter, and N. F. Reuel. 2018. "Substrate-Wrapped, Single-Walled Carbon Nanotube Probes for Hydrolytic Enzyme Characterization." *Analytical Chemistry* 90, no. 8: 5209–5216. https://doi.org/10.1021/acs.analchem.7b05444.

Kim, D.-H., D. Jeong, K.-Y. Song, and K.-H. Seo. 2018. "Comparison of Traditional and Backslopping Methods for Kefir Fermentation Based on Physicochemical and Microbiological Characteristics." *LWT* 97: 503–507. https://doi.org/10.1016/j.lwt.2018.07.023.

Kuo, M.-T., J. F. Raffaelle, E. M. Waller, et al. 2023. "Screening Enzymatic Degradation of Polyester Polyurethane With Fluorescent Single-Walled Carbon Nanotube and Polymer Nanoparticle Conjugates." *ACS Nano* 17, no. 17: 17021–17030. https://doi.org/10.1021/acsnano.3c04347.

Kuo, M.-T., and N. F. Reuel. 2025. "Resolving the Kinetics of Single-Walled Carbon Nanotube–Polyester Polyurethane Nanoparticle Conjugate Fluorescence Sensors Toward Polymer Degrading Enzymes." *Nano Letters* 25, no. 2: 715–721. https://doi.org/10.1021/acs.nanolett.4c04964.

Lobry, J. R., J. P. Flandrois, G. Carret, and A. Pave. 1992. "Monod's Bacterial Growth Model Revisited." *Bulletin of Mathematical Biology* 54, no. 1: 117–122. https://doi.org/10.1016/S0092-8240(05)80179-X.

Ma, Y., Z. Wang, I. Castillo, et al. 2021. "Reinforcement Learning-Based Fed-Batch Optimization with Reaction Surrogate Model." In *2021 American Control Conference (ACC)*, 2581–2586. New Orleans, LA, USA: IEEE. https://doi.org/10.23919/ACC50511.2021.9482807.

Mannaa, M., G. Han, Y.-S. Seo, and I. Park. 2021a. "Evolution of Food Fermentation Processes and the Use of Multi-Omics in Deciphering the Roles of the Microbiota." *Foods* 10, no. 11: 2861. https://doi.org/10.3390/foods10112861.

Mannaa, M., G. Han, Y.-S. Seo, and I. Park. 2021b. "Evolution of Food Fermentation Processes and the Use of Multi-Omics in Deciphering the Roles of the Microbiota." *Foods* 10, no. 11: 2861. https://doi.org/10.3390/foods10112861.

Mears, L., S. M. Stocks, G. Sin, and K. V. Gernaey. 2017. "A Review of Control Strategies for Manipulating the Feed Rate in Fed-Batch Fermentation Processes." *Journal of Biotechnology* 245: 34–46. https://doi.org/10.1016/j.jbiotec.2017.01.008.

Metwally, M. 1997. "Glucoamylase Production in Continuous Cultures of Aspergillus Niger With Special Emphasis on Growth Parameters." *World Journal of Microbiology and Biotechnology* 14, no. 1: 113–118. https://doi.org/10.1023/A:1008840904246.

Rajasekhar, N., T. K. Radhakrishnan, and S. N. Mohamed. 2024. "Reinforcement Learning Based Temperature Control of a Fermentation Bioreactor for Ethanol Production." *Biotechnology and Bioengineering* 121, no. 10: 3114–3127. https://doi.org/10.1002/bit.28784.

Salvato, E., G. Fenu, E. Medvet, and F. A. Pellegrino. 2021. "Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning." *IEEE Access* 9: 153171–153187. https://doi.org/10.1109/ACCESS.2021.3126658.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. 2017. "Proximal Policy Optimization Algorithms." *arXiv* 1: 6347. https://doi.org/10.48550/ARXIV.1707.06347.

Silver, D., A. Huang, C. J. Maddison, et al. 2016. "Mastering the Game of Go With Deep Neural Networks and Tree Search." *Nature* 529, no. 7587: 484–489. https://doi.org/10.1038/nature16961.

Teo, K. T. K., H. J. Tham, and M. K. Tan. 2010. "Optimization of Substrate Feed Flow Rate for Fed-Batch Yeast Fermentation Process." In *2010 Second International Conference on Computational Intelligence, Modelling and Simulation*, 233–238. Bali, Indonesia: IEEE. https://doi.org/10.1109/CIMSiM.2010.94.

Treloar, N. J., A. J. H. Fedorec, B. Ingalls, and C. P. Barnes. 2020a. "Deep Reinforcement Learning for the Control of Microbial Co-Cultures in Bioreactors." *PLoS Computational Biology* 16, no. 4: e1007783. https://doi.org/10.1371/journal.pcbi.1007783.

Treloar, N. J., A. J. H. Fedorec, B. Ingalls, and C. P. Barnes. 2020b. "Deep Reinforcement Learning for the Control of Microbial Co-Cultures in Bioreactors." *PLoS Computational Biology* 16, no. 4: e1007783. https://doi.org/10.1371/journal.pcbi.1007783.

Yerolla, R., S. H. Nekkanti, S. Pullanikkattil, and C. S. Besta. 2023. "Modeling and Control of Beer Fermentation for Optimal Flavor and Performance." *Chemical Engineering & Technology* 46, no. 8: 1554–1565. https://doi.org/10.1002/ceat.202200376.

Zhang, H., H. Chen, C. Xiao, et al. Robust Deep Reinforcement Learning against Adversarial Perturbations on State Observations. 2020. https://doi.org/10.48550/ARXIV.2003.08938.

Zhang, T., X. Wang, B. Liang, and B. Yuan Catastrophic Interference in Reinforcement Learning: A Solution Based on Context Division and Knowledge Distillation. 2021. https://doi.org/10.48550/ARXIV.2109.00525.

Supporting Information

Additional supporting information can be found online in the Supporting Information section.
Supporting File: bit70234-sup-0001-RL_Controlled_reactor_SI.docx.